

25SC2107E: Machine Learning (Skilling) — Experiment 2 Report

Titanic Survival — Preprocessing Pipeline and Cleaned Dataset

Course Outcome: CO1 | Environment: VS Code | Faculty: Dr. Omprakash Gottam

1. Executive Summary & Aim

The objective of Experiment 2 is to transform raw tabular data from the Titanic survival dataset into a clean, normalized numerical array ready for Machine Learning algorithms. A raw machine learning table cannot directly process missing values (NaNs), unstructured text categories ('male'/'female'), or unscaled numerical ranges (\$0 to \$512 vs 0 to 8). In this lab, we build an end-to-end reproducible preprocessing pipeline using scikit-learn's Pipeline and ColumnTransformer, enforcing proper train/validation data splitting to eliminate data leakage and saving the fitted pipeline as a serialized joblib object.

2. Data Cleaning & Anti-Data Leakage Strategy

Data Leakage Prevention: Machine learning models must evaluate on unseen validation data under realistic conditions. If global statistics (e.g., median age or feature scaling parameters) are calculated across the entire dataset prior to splitting, information from validation rows leaks into the training pipeline. Therefore, our pipeline strictly enforces:

- **Step 1: Drop Leaking & Redundant Columns:** alive (exact target duplicate), who, adult_male, class, embark_town (duplicate kept fields), deck (688/891 missing values), and alone (manually rebuilt).
- **Step 2: Feature Engineering & Verification:** Rebuilt is_alone = ((sibsp + parch) == 0).astype(int) and verified 100% agreement against original ground truth.
- **Step 3: Train-Test Split Before Preprocessing:** Executed 80/20 stratified split (random_state=42, stratify=y) yielding 712 train rows and 179 validation rows.
- **Step 4: Fit on Train Only:** Call preprocessor.fit_transform(X_train) and preprocessor.transform(X_val).

3. Feature Transformation Matrix (8 Raw → 11 Clean Columns)

Group	Raw Column	Output Cols	Imputation Strategy	Transformation Machine	Why / Rationale
Numeric	age	1	Median (28.0)	StandardScaler	Robust against skewed ages; centers mean=0, std=1
Numeric	fare	1	Median (14.45)	StandardScaler	Eliminates scale dominance of \$0-\$512 vs 0-8
Numeric	sibsp	1	Median (0.0)	StandardScaler	Rescales count of siblings/spouses
Numeric	parch	1	Median (0.0)	StandardScaler	Rescales count of parents/children
Numeric	is_alone	1	Median (1.0)	StandardScaler	Engineered binary indicator (0 or 1)
Categorical	sex	2	Most Frequent	OneHotEncoder	Produces sex_female, sex_male (nominal)
Categorical	embarked	3	Most Frequent ('S')	OneHotEncoder(ignore)	Produces embarked_C, Q, S; ignores unseen
Ordinal	pclass	1	Most Frequent (3)	OrdinalEncoder([1,2,3])	Preserves intrinsic rank order (1st > 2nd > 3rd)
TOTAL	8 Columns	11 Columns	—	ColumnTransformer	Net growth from One-Hot Encoding (+3 cols)

4. Preprocessing & Feature Distribution Analysis

Figure 1 illustrates the exact impact of standard scaling on the fare feature. Notice that scaling changes only the units (Z-score along the horizontal axis), leaving the underlying distribution shape completely unaltered.

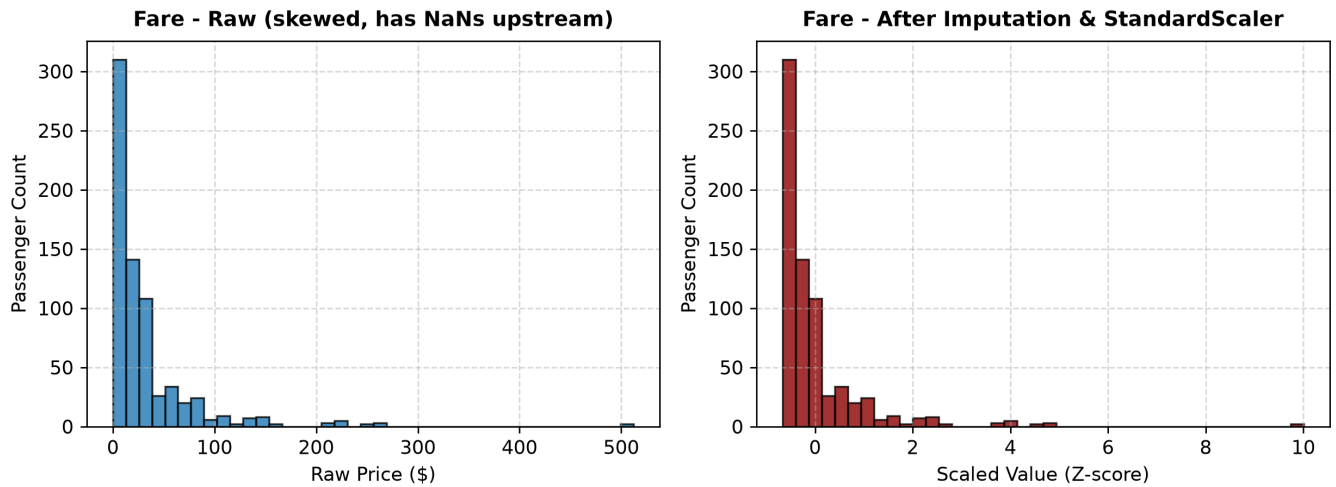
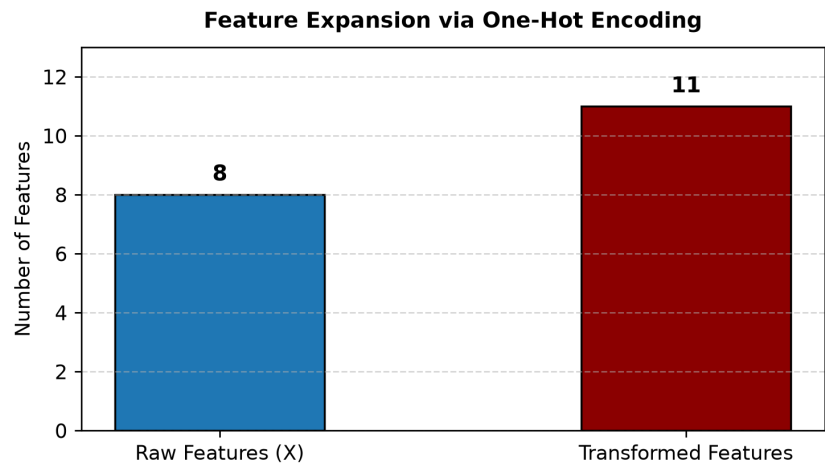


Figure 2 shows the expansion from 8 input columns to 11 model-ready numerical columns due to One-Hot Encoding of nominal categorical variables.



5. Console Execution Logs & Pipeline Validation

```
is_alone matches the original alone column: True
X_train shape: (712, 8) | X_val shape: (179, 8)
Columns before vs after preprocessing: 8 -> 11
Pipeline validation PASSED
Features: 11 Train rows: 712
Fitted pipeline successfully saved to titanic_preprocessor.joblib
```

6. Self-Exploration & Viva Voce Solutions

Q1: You dropped alive but kept survived. Both say the same thing. Why is one a cheat and the other the answer we want?

Answer: survived is the target variable (y) we wish to predict. alive is a literal text string ('yes'/'no') representing the exact same fact. Keeping alive as an input feature in X introduces a catastrophic *data leak*: the machine learning model would simply learn the trivial rule $\text{alive} == \text{'yes'} \rightarrow \text{survived} = 1$, achieving 100% training accuracy without discovering any real underlying survival patterns.

Q2: A classmate scales the whole dataset first and splits afterwards. Their score is higher than yours. Where did the extra marks come from, and why are they not real?

Answer: Scaling before splitting causes *data leakage* from the validation fold into the training calculations. StandardScaler computes the global mean (μ) and standard deviation (σ). If calculated over all 891 passengers, the training set scaling includes information from the validation set (the exam paper). The resulting high score is an artificial, overoptimistic illusion that will fail on real production data.

Q3: age is filled with the median, not the average. Why?

Answer: Passenger age has a skewed distribution containing extreme values (infants vs elderly passengers). The arithmetic mean is highly sensitive to outliers and skewness, whereas the median is robust to extreme values, accurately capturing the true central tendency of the data without distortion.

Q4: pclass uses OrdinalEncoder but embarked uses OneHotEncoder. What is it about the data that decides which one is correct?

Answer: pclass possesses a natural numerical hierarchy ($1^{\text{st}} > 2^{\text{nd}} > 3^{\text{rd}}$ class socio-economic standing), so OrdinalEncoder preserves rank order (0, 1, 2). Conversely, embarked represents nominal ports (Cherbourg, Queenstown, Southampton) without any inherent mathematical order. Using OrdinalEncoder on nominal ports would falsely imply $S < C < Q$, introducing artificial mathematical bias. Therefore, OneHotEncoder is required for nominal categories.

Q5: You started with 8 columns and ended with 11. Explain where each of the 3 extra columns came from.

Answer: The 8 raw input columns split into: 5 Numeric ('age', 'fare', 'sibsp', 'parch', 'is_alone'), 1 Ordinal ('pclass'), and 2 Categorical ('sex', 'embarked').

- sex (2 categories: female, male) $\rightarrow$ 2 one-hot columns ('sex_female', 'sex_male').
- embarked (3 categories: C, Q, S) $\rightarrow$ 3 one-hot columns ('embarked_C', 'embarked_Q', 'embarked_S').

Original 2 categorical columns are replaced by 5 binary columns ($5 - 2 = +3$ extra columns). Thus, $8 + 3 = 11$ total columns.

Q6: Why was alone dropped and then rebuilt as is_alone? What did the check in Cell 3 prove?

Answer: alone was dropped to practice feature engineering from fundamental raw components ('sibsp' and 'parch'). Rebuilding $\text{is_alone} = (\text{sibsp} + \text{parch}) == 0$.astype(int) and asserting ($\text{df['is_alone']} == \text{titanic['alone']}.astype(int)$).all() proved 100% mathematical accuracy against ground truth.

Q7: Explore: Change handle_unknown='ignore' to 'error', then pass a passenger whose port is 'X'. What happens?

Answer: With handle_unknown='error', scikit-learn raises ValueError: Found unknown categories ['X'] in column 1 during transform. In production, handle_unknown='ignore' prevents crashes by outputting all zeros for unencountered categories.

Q8: Explore: Replace StandardScaler with MinMaxScaler and draw Cell 10 again. What changed, and what stayed the same?

Answer: Visual comparison below demonstrates that the distribution shape and skewness remain identical. What changes is the numerical domain: StandardScaler scales data to mean=0, std=1 (Z-scores), whereas MinMaxScaler compresses all values strictly into $[0, 1]$.

Figure 3: Empirical Comparison of StandardScaler vs MinMaxScaler (Viva Q8 Output)

